

Unidad 7.- PROGRAMACIÓN DE UN SGBD.

Unidad 8.- ADMINISTRACIÓN DE UN SGBD.

RA5. Desarrolla procedimientos almacenados evaluando y utilizando las sentencias del lenguaje incorporado en el sistema gestor de bases de datos.

EJERCICIO 1 (0.5 ptos)

a) Se han identificado las diversas formas de automatizar tareas. 5%

Identifique las diversas formas de automatizar tareas en un párrafo elaborado.

EJERCICIO 2 (0.5 ptos)

b) Se han reconocido los métodos de ejecución de guiones. 5%

Indique cuales son los principales métodos de ejecución de guiones en un párrafo elaborado.

EJERCICIO 3 (0.5 ptos)

c) Se han identificado las herramientas disponibles para editar guiones.5%

Identifique las herramientas disponibles para editar guiones en un párrafo elaborado.

EJERCICIO 4 (1 pto)

d) Se han definido y utilizado guiones para automatizar tareas.10%

"Crea un procedimiento almacenado llamado **ContarProductosPorLinea**. Este debe recibir un parámetro de entrada con el nombre de la línea (p_productLine) y devolver el total de productos a través de un parámetro de salida (p_total), consultando la tabla products y filtrando los resultados según el parámetro proporcionado."

-- Para ejecutarlo:

```
-- CALL ContarProductosPorLinea('Classic Cars',@Num_productos));  
SELECT @Num_productos;
```

EJERCICIO 5 (2 ptos)

- e) Se ha hecho uso de las funciones proporcionadas por el sistema gestor.5%
- f) Se han definido procedimientos y funciones de usuario.10%
- j) Se han utilizado excepciones.5%

"Crea un procedimiento almacenado llamado **ResumenVentasCliente** que reciba como parámetro de entrada el número de identificación de un cliente (p_customerNumber) y devuelva, a través de cuatro parámetros de salida, los siguientes datos:

1. Nombre del cliente (p_nombre).
2. Total de pagos realizados (p_cantidadPagos).
3. Suma acumulada de los pagos (p_sumaPagos).
4. Promedio de los pagos realizados (p_promedioPagos).

El procedimiento debe realizar una unión (INNER JOIN) entre las tablas customers y payments para calcular y devolver un resumen financiero del cliente que incluya:

1. el nombre del cliente,
2. la cantidad total de pagos realizados
3. la suma acumulada de todos sus pagos
4. el promedio de los pagos realizados
5. Además, debe incluir un control de errores: si el cliente no existe en la tabla customers, se deberá lanzar una excepción utilizando SIGNAL SQLSTATE '45000' con un mensaje de error personalizado SET MESSAGE_TEXT.

-- Para ejecutarlo:

```
-- CALL ResumenVentasCliente(103, @Nomcli, @NumPagos, @totalPagos,  
@PromedioPagos);  
select @Nomcli, @NumPagos, @totalPagos, @PromedioPagos;
```

EJERCICIO 6 (2 ptos)

- e) Se ha hecho uso de las funciones proporcionadas por el sistema gestor.5%
- f) Se han definido procedimientos y funciones de usuario.10%
- g) Se han utilizado estructuras de control de flujo.5%

Crear una función almacenada en MySQL llamada **EvaluarPrecioProducto** que permita determinar la posición competitiva del precio de un producto específico en relación con el promedio de su misma línea de productos.

Requerimientos funcionales:

1. Entrada: La función debe recibir como parámetro el código de un producto (p_productCode).
2. Lógica de Agregación: La función debe calcular internamente el precio de compra promedio (AVG) de todos los productos que pertenecen a la misma categoría (productLine) del producto consultado.
3. Estructura de Control: Mediante el uso de bloques IF-ELSE, se debe comparar el precio de compra (buyPrice) del producto ingresado contra el precio de compra promedio calculado de su categoría (productLine).
4. Salida: La función debe retornar una cadena de texto (VARCHAR) con uno de los siguientes diagnósticos:
 1. *"Precio superior al promedio"*: Si el precio del producto es mayor a la media de su línea.
 2. *"Precio inferior al promedio"*: Si el precio del producto es menor a la media de su línea.
 3. *"Precio exacto al promedio"*: En el caso de que ambos valores coincidan.

-- Ejemplo de uso:

```
-- SELECT productCode, productName, buyPrice, EvaluarPrecioProducto(productCode)
-- FROM products;
```

EJERCICIO 7 (1.5 ptos)

- h) Se han definido eventos y disparadores.10%
- j) Se han utilizado excepciones.5%

Se necesita crear un TRIGGER denominado **tg_validar_pago_antes_insert BEFORE INSERT** en la tabla payments que valide que el monto (valor) del pago sea mayor a cero y que la fecha de pago no puede ser futura (posterior a la fecha actual). Utilizaremos un control de errores con SIGNAL SQLSTATE '45000' para detener la inserción y mostrar un mensaje personalizado con SET MESSAGE_TEXT.

Este enfoque es mejor que simplemente corregir el dato, ya que obliga al usuario o a la aplicación a enviar datos correctos.

Nota: La función **CURDATE()** en MySQL devuelve la **fecha actual** del sistema en el que se ejecuta el servidor.

EJERCICIO 8 (1 pto)

- h) Se han definido eventos y disparadores.10%

Contexto: Se proporciona un evento que actualice un estado (por ejemplo, marcar pedidos antiguos como "Archivado") e incluya un control de errores básico que detenga la ejecución si algo falla. Dicho evento se ejecuta cada 24 horas.

Actualmente, el código carece de documentación, es necesario poner los comentarios dónde se indica:

--1. **Aquí irá su comentario**

SET GLOBAL event_scheduler = ON;

DELIMITER //

--2. **Aquí irá su comentario**

CREATE EVENT evt_archivar_pedidos_simples

ON SCHEDULE EVERY 1 DAY

DO

BEGIN

--3. **Aquí irá su comentario**

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

-- Aquí termina la ejecución si falla algo

END;

--4. Lógica sencilla: [Aquí irá su comentario](#)

UPDATE orders

SET status = 'Archived'

WHERE orderDate < DATE_SUB(CURDATE(), INTERVAL 1 YEAR)

AND status = 'Shipped';

END //

DELIMITER ;

Nota: `DATE_SUB(CURDATE( ), INTERVAL 1 YEAR)` calcula exactamente la fecha de hace un año respecto al día de hoy.

EJERCICIO 9 (1 pto)

i) Se han utilizado cursores.10%

Contexto: Se te proporciona el código de un procedimiento almacenado diseñado para procesar registros de forma secuencial. Actualmente, el código carece de documentación y su estructura lógica debe ser validada.

Tu tarea:

Analiza el siguiente bloque de código y realiza las siguientes acciones:

1) **Identificación de Componentes:** Inserta comentarios en el código utilizando la sintaxis `--` indicando exactamente dónde se produce cada uno de los siguientes hitos:

- **Declaración de variables** de apoyo.
- **Declaración del cursor** (definiendo la consulta SELECT).
- **Manejador de error (Handler):** Identifica la línea que gestiona el final del bucle mediante la condición `NOT FOUND`.
- **Apertura del cursor.**
- **Lógica de programación:** Localiza el bucle de lectura (`FETCH`) y el procesamiento de los datos.
- **Cierre del cursor.**

Esta función recibe el nombre de un país y un umbral de crédito y devuelve la suma total procesada fila a fila. Esta función recorre los clientes de un país específico mediante un cursor para sumar el límite de crédito (`creditLimit`), pero solo de aquellos que superen una cifra determinada.

```
DELIMITER //
```

```
CREATE FUNCTION calcular_credito_pais(p_pais VARCHAR(50), p_umbral
DECIMAL(10,2))
RETURNS DECIMAL(15,2)
BEGIN
```

```
-- 1. Aquí irá su comentario
```

```
DECLARE done INT DEFAULT FALSE;
DECLARE v_credito_actual DECIMAL(10,2);
DECLARE v_suma_total DECIMAL(15,2) DEFAULT 0;
```

```
-- 2. Cursor: Aquí irá su comentario
```

```
DECLARE cur_clientes CURSOR FOR
  SELECT creditLimit
  FROM customers
  WHERE country = p_pais;
```

```
-- 3. Aquí irá su comentario
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
```

```
-- 4. Aquí irá su comentario
```

```
OPEN cur_clientes;
```

```
-- 5. Aquí irá su comentario
```

```
read_loop: LOOP
  FETCH cur_clientes INTO v_credito_actual;
```

```
  IF done THEN
    LEAVE read_loop;
```

```
  END IF;
```

```
--5.1. Aquí irá su comentario
```

```
  IF v_credito_actual > p_umbral THEN
    SET v_suma_total = v_suma_total + v_credito_actual;
  END IF;
```

```
END LOOP;
```

```
-- 6. Aquí irá su comentario
```

```
CLOSE cur_clientes;
```

```
  RETURN v_suma_total;
```

```
END //
```

```
DELIMITER ;
```

INFORMACIÓN DE INTERÉS

Disponibilidad de los modificadores OLD y NEW

Trigger Event	OLD	NEW
INSERT	No	Yes
UPDATE	Yes	Yes
DELETE	Yes	No

ORDEN DE COMANDOS SQL COMANDOS BÁSICOS SQL



FORMATO DE ENTREGA:

Fichero **nombre_apellido.PDF** con el nombre y apellido del estudiante.
Se calificará sobre el **PDF** entregado.

Ejercicios ordenados, **sin el enunciado**, enumerados con la respuesta:

- Si un ejercicio NO se responde, deberá aparecer con su número.
- Ejercicios teoría (1, 2, 3): texto.
- Ejercicios (8, 9): solo las líneas ordenadas de comentarios, SIN el código.
- Resto de ejercicios (4, 5, 6, 7) solo el script SQL, deberá aparecer en el **formato de ejemplo** que se muestra a continuación:

Formato de ejemplo script SQL:

Se escribirá el código **indentado y ordenado como en el ejemplo, sin color de letra, sin color de fondo ni líneas en blanco.**

No se utilizarán alias para ninguna tabla y no se utilizará la tabla para referenciar una columna.

EJERCICIO 7

```
DELIMITER $$
```

```
CREATE TRIGGER trigger_name  
  BEFORE INSERT  
  ON table_name FOR EACH ROW  
BEGIN  
  -- Código irá aquí...  
END$$
```

```
DELIMITER ;
```